

Job Board Platform

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This document describes what the Job Board Platform does and how it works, as the system is actually built and running today. It is meant to be a clear, easy-to-read reference for anyone reviewing, maintaining, or continuing development on the project.

1.2 Scope

The Job Board Platform is a web application that connects job seekers with companies. Job seekers can register, build a profile, browse and filter job listings, and apply to jobs. Companies can register, wait for admin approval, and once approved, post and manage job listings and review applications. Admins verify new company accounts.

1.3 Intended Audience

This document is written for the project owner, the internship lead reviewing the project, and anyone who may work on or review the codebase later.

2. Overall Description

2.1 Product Perspective

The system is a full-stack web application. The frontend is built with React, the backend is built with Python and FastAPI, and the database is PostgreSQL. The frontend and backend are deployed on Vercel, and the database is hosted separately on Supabase.

2.2 User Classes

- Job Seeker: a registered user who can browse jobs, build a profile with skills, and submit applications.
- Company Representative: a registered user whose company account starts as pending and gains posting ability only after an admin approves it. The same login is used for both their personal job-seeker activity and their company management, following the same model as LinkedIn.
- Admin: a user with an `is_admin` flag set to true, who can verify pending company accounts and manage users.

2.3 Operating Environment

The system runs entirely in a standard web browser, with no local installation needed by end users. It depends on an internet connection to reach the deployed backend and database.

2.4 Constraints

- Built and maintained by a single developer, which limits how much can be delivered in the current timeframe.
- Not built for high traffic or large-scale concurrent use at this stage; caching and heavier scaling infrastructure are deliberately left for later, once real usage justifies them.

3. Functional Requirements

3.1 Authentication

- A user can register with a name, email, and password. Passwords are hashed with bcrypt before being stored; they are never stored or shown in plain text.
- A user can log in with their email and password. On success, the backend issues a JWT access token, valid for 24 hours.
- The frontend stores this token in the browser's local storage and attaches it to future requests to prove the user is logged in.
- There is currently no refresh token and no way to revoke a token before it expires on its own.

3.2 Authorization

- Access is controlled through a small set of roles: job seeker, company, and admin, combined with ownership checks on specific resources.
- A job seeker can register, add skills to their profile, browse and filter jobs, and apply to jobs, with a rule preventing the same job from being applied to twice.
- A company account is inactive/unverified until an admin approves it. Only after approval can it post, edit, or delete its own job listings.

- A company can only manage its own job listings, never another company's.
- An admin is identified purely by an `is_admin` flag on their user account, not by a separate admin account type.

3.3 Company Registration and Verification

- A user can register a company under the same account they already use as a job seeker, rather than creating a separate account.
- The company stays in a pending state until an admin reviews and approves it.
- Once approved, the same login can manage both the user's personal job-seeker activity and their company's job postings.

3.4 Job Management

- An approved company can create, edit, and delete its own job postings.
- Each job posting includes a title, description, location, salary range, and employment type.
- Job seekers can browse all open job postings, and filter them by fields such as location, employment type, and salary.

3.5 Applications

- A job seeker can apply to any open job, but cannot apply to the same job more than once.
- An application moves through a status workflow, tracked from submission through to a final outcome such as accepted, rejected, or hired.
- A job seeker can view the status of their own applications.
- A company can view and update the status of applications submitted to its own job postings only.

3.6 Admin Functions

- An admin can view the list of all registered users.
- An admin can review and approve pending company accounts.
- An admin can deactivate a user account.

4. Non-Functional Requirements

4.1 Security

- Passwords are always hashed, never stored or returned in plain text.
- Every protected action checks both the user's role and, where relevant, whether they own the specific resource being acted on.
- Configuration and secrets (database URL, JWT signing key) are read from environment variables, not hardcoded in the source code.

4.2 Data Storage

- All data is stored in a PostgreSQL database, managed through SQLAlchemy as the ORM and Alembic for schema migrations.
- Simple, everyday database operations go through the ORM; complex search and filtering queries use raw SQL for better performance.

4.3 API Design

- The backend exposes a REST API, versioned under a `/api/v1` path.
- Every endpoint follows consistent conventions for pagination, filtering, sorting, and error responses, so the frontend can rely on predictable behavior across the whole API.

4.4 Testing

- Unit tests cover business logic in the service layer, such as role and ownership checks, using fake data rather than a real database.
- Integration tests cover repository and database behavior, particularly the raw SQL search queries.
- A small, deliberately limited set of end-to-end tests covers only the most critical flows: registration, applying to a job, and company verification.

4.5 Deployment

- The frontend and backend are deployed together on Vercel; the database is hosted separately on Supabase.
- The database connects through Supabase's pooled connection, suited to the short-lived, serverless nature of the Vercel deployment.

5. External Interface Requirements

5.1 User Interface

The frontend follows a single, consistent visual theme across every page: dark slate as the primary color, teal as the accent color, and a light slate background, applied consistently to the job listings page, job details, dashboards, and admin views.

5.2 API Interface

The frontend communicates with the backend exclusively through the REST API described in Section 4.3, using JSON for all requests and responses.

6. Known Deviations from the Original Design

The system was originally planned with a more advanced authentication model, including OAuth login and a short-lived access token paired with a longer-lived, revocable refresh token. During development, this was deliberately simplified to a single JWT with email and password login, since it was easier to build correctly within the project's timeline. This section exists so the gap between the original plan and the current system is clear and intentional, rather than something to be rediscovered by accident later.

- No refresh token currently exists; a stolen or leaked access token cannot be revoked early and remains valid for its full 24-hour lifetime.
- The access token is stored in the browser's local storage rather than in memory, which is a known, accepted tradeoff at the current stage rather than the original design.
- The application status workflow in the running system is more detailed than what was first documented, and should be reviewed to confirm it reflects an intentional decision rather than unplanned scope creep.